

# ArrayList Review: Trace These

For each problem, trace the code by hand. Record the state of every variable and list after each step, and write down exactly what is printed (or the final contents of the list). If the code throws an exception or misbehaves, say where and why.

Problem numbers match the original ArrayList Practice 1 booklet.

---

## Problem 4

Trace the following code. Keep track of `k`, the value returned by `students.set(...)`, and the contents of `students` after each iteration of the first loop. Then trace the second loop.

```
List<String> students = new ArrayList<String>();

students.add("Alex");
students.add("Bob");
students.add("Carl");

for (int k = 0; k < students.size(); k++)
{
    System.out.print(students.set(k, "Alex") + " ");
}

System.out.println();

for (String str : students)
{
    System.out.print(str + " ");
}
```

Write out both lines of output exactly as they would appear.

| k | students before | value returned by set | students after | printed so far |
|---|-----------------|-----------------------|----------------|----------------|
| 0 |                 |                       |                |                |
| 1 |                 |                       |                |                |
| 2 |                 |                       |                |                |

---

## Problem 7

Trace a call to `match` with `numList = [5, 2, 10, 20, 16]` and `key = 5`.

```
public static ArrayList<Integer> match(ArrayList<Integer> numList, int key)
{
    ArrayList<Integer> returnList = new ArrayList<Integer>();

    int i = 0;
    while (i < numList.size())
    {
        int num = numList.get(i);
        if (num % key == 0)
        {
            numList.remove(i);
            returnList.add(num);
        }
        i++;
    }
    return returnList;
}
```

The method is *intended* to leave `[2, 16]` in `numList` and return `[5, 10, 20]`.

1. Fill in a trace table with one row per loop iteration.
2. Compare the final `numList` and the returned list to the intended results.
3. Identify the iteration where things first go wrong, and explain in one sentence why.

## Problem 8

Trace `mystery(values)`, where `values` initially holds `[0, 0, 4, 2, 5, 0, 3, 0]`.

```
public static void mystery(List<Integer> nums)
{
    for (int k = 0; k < nums.size(); k++)
    {
        if (nums.get(k).intValue() == 0)
        {
            nums.remove(k);
        }
    }
}
```

Record `k`, `nums.size()`, `nums.get(k)`, whether the element is removed, and the contents of `nums` at the end of each iteration. What are the final contents of `values`? Does the method do what its structure suggests it should? If not, which elements were never examined?

| k | nums.size() | nums.get(k) | removed? | nums after |
|---|-------------|-------------|----------|------------|
|---|-------------|-------------|----------|------------|

---

---

## Problem 9

Trace `removeDups` when `myData` holds `3 3 4 4 4 8 7 7 7`.

```
private ArrayList myData;

public void removeDups()
{
    int k = 1;
    while (k < myData.size())
    {
        if (myData.get(k).equals(myData.get(k - 1)))
        {
            myData.remove(k);
        }
        k++;
    }
}
```

The method is *intended* to leave 3 4 8 7.

1. Trace the loop, recording k, the two elements being compared, whether a removal happens, and the contents of myData at the end of each iteration.
2. What are the final contents of myData?
3. Which duplicates survived, and what about the way k advances after a removal explains it?

k   myData.get(k)   myData.get(k - 1)   equal?   myData after

## Problem 11

Trace the following code, keeping track of the contents of both lists after every statement. Pay close attention to what remove(1) returns and to which indices are valid at the moment get(2) is called.

```
ArrayList<Integer> oldList = new ArrayList();
oldList.add(100);
oldList.add(200);
oldList.add(300);
oldList.add(400);
```

```
ArrayList<Integer> newList = new ArrayList();
newList.add(oldList.remove(1));
newList.add(oldList.get(2));
System.out.println(newList);
```

| After statement                | oldList | newList |
|--------------------------------|---------|---------|
| oldList.add(400)               |         |         |
| newList.add(oldList.remove(1)) |         |         |
| newList.add(oldList.get(2))    |         |         |

What is printed? Does the code compile and run without error?